Patrones UX

Estados y Feedback

📝 2026-01 — ⏱️ 25 min.

Los 4 Estados Clave

Toda Pantalla Tiene 4 Estados

Cuando diseñas una pantalla, debes diseñar los **4 estados posibles**:

1. **Loading** — Los datos están cargando
2. **Empty** — No hay datos que mostrar
3. **Error** — Algo salió mal
4. **Success / Populated** — El estado normal con datos

Un error muy común: solo diseñar el estado "success" y olvidar los otros tres.

```
function ListaUsuarios() {
  if (isLoading) return <EstadoCarga />   // ← diseñar esto
  if (error)     return <EstadoError />   // ← diseñar esto
  if (!usuarios.length) return <EstadoVacio /> // ← diseñar esto
  return <TablaUsuarios datos={usuarios} />    // ← el estado "feliz"
}
```

Estados de Carga

Spinner vs Skeleton

Spinner (Loading indicator):

```html
<div className="spinner" role="status" aria-label="Cargando...">
  <span className="sr-only">Cargando...</span>
</div>
```

```css
.spinner {
  width: 40px; height: 40px;
  border: 3px solid hsl(220, 13%, 91%);
  border-top-color: var(--color-accion);
  border-radius: 50%;
  animation: girar 0.7s linear infinite;
}
@keyframes girar { to { transform: rotate(360deg); } }

/* Ocultar texto del span pero mantenerlo para lectores de pantalla */
.sr-only {
  position: absolute; width: 1px; height: 1px;
  overflow: hidden; clip: rect(0,0,0,0);
}
```

✅ Bueno para acciones rápidas (< 2 segundos) ⚠️ Poco informativo para cargas largas

Skeleton Screens

Los skeletons muestran la forma aproximada del contenido mientras carga. Reducen la percepción de espera.

```html
<div class="card-skeleton">
  <div class="skeleton skeleton--imagen"></div>
  <div class="skeleton skeleton--titulo"></div>
  <div class="skeleton skeleton--texto"></div>
  <div class="skeleton skeleton--texto skeleton--corto"></div>
</div>
```

```css
.skeleton {
  background: hsl(220, 13%, 91%);
  border-radius: 4px;
  animation: pulso 1.5s ease-in-out infinite;
}
.skeleton--imagen  { height: 200px; margin-bottom: 12px; }
.skeleton--titulo  { height: 24px; width: 70%; margin-bottom: 8px; }
.skeleton--texto   { height: 16px; margin-bottom: 6px; }
.skeleton--corto   { width: 50%; }

@keyframes pulso {
  0%, 100% { opacity: 1; }
  50%      { opacity: 0.5; }
}
```

Estado Vacío

Diseñar el Estado Vacío

El estado vacío aparece cuando no hay datos que mostrar. Es una oportunidad de guiar al usuario.

Componentes de un buen estado vacío:

1. Ilustración o icono descriptivo
2. Título claro que explica la situación
3. Descripción corta con contexto
4. Llamada a la acción para resolver la situación

Diseñar el Estado Vacío

```
function EstadoVacio() {
  return (
    <div className="estado-vacio">
      <img src="/img/bandeja-vacia.svg" alt="" aria-hidden="true" />
      <h2>No tienes notificaciones</h2>
      <p>Cuando tengas actividad nueva, aparecerá aquí.</p>
    </div>
  )
}
```

❌ **No hagas esto:** mostrar solo un texto "No hay datos" sin contexto.

✅ **Haz esto:** explica *por qué* está vacío y *qué puede hacer* el usuario.

Estado de Error

Errores Útiles

Un mensaje de error útil tiene 3 partes:

1. **Qué salió mal** — sin tecnicismos
2. **Por qué** — si es relevante para el usuario
3. **Qué hacer** — siguiente paso accionable

Errores Útiles

```
// ❌ Error inútil
<p>Error 503</p>
<p>Something went wrong.</p>

// ✅ Error útil
<div role="alert" className="error-panel">
  <h2>No pudimos cargar los resultados</h2>
  <p>
    Hubo un problema al conectar con el servidor.
    Tu conexión a internet puede estar fallando.
  </p>
  <button onClick={reintentar}>Intentar de nuevo</button>
  <a href="/soporte">Contactar soporte</a>
</div>
```

Tono: empático, no técnico, sin culpar al usuario. "Algo salió mal" mejor que "Error del usuario".

Feedback de Éxito y Toasts

Toasts / Notificaciones

Los toasts son notificaciones temporales que aparecen en un corner de la pantalla.

```
function Toast({ tipo, mensaje, onCerrar }) {
  useEffect(() => {
    const timer = setTimeout(onCerrar, 4000)
    return () => clearTimeout(timer)
  }, [onCerrar])

  return (
    <div className={`toast toast--${tipo}`} role="status" aria-live="polite">
      <span className="toast__icono">
        {tipo === 'exito' ? '✅' : tipo === 'error' ? '❌' : 'ℹ️'}
      </span>
      <p className="toast__mensaje">{mensaje}</p>
      <button className="toast__cerrar" aria-label="Cerrar notificación" onClick={onCerrar}>
        ×
      </button>
    </div>
  )
}
```

Posicionamiento y Duración de Toasts

```
.toast-container {
  position: fixed;
  bottom: 24px;
  right: 24px;
  z-index: 9999;
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.toast {
  min-width: 280px;
  max-width: 400px;
  padding: 12px 16px;
  border-radius: 8px;
  display: flex;
  align-items: center;
  gap: 12px;
  box-shadow: 0 4px 12px hsl(0, 0%, 0%, 0.15);
  animation: deslizar-entrada 0.3s ease;
}

.toast--exito { background: hsl(142, 76%, 97%); border-left: 4px solid hsl(142, 71%, 45%); }
.toast--error  { background: hsl(0, 93%, 97%);  border-left: 4px solid hsl(0, 84%, 60%); }
```

Resumen — Estados y Feedback

- ✅ Diseña siempre los **4 estados**: loading, empty, error, success
- ✅ Usa **skeleton screens** para cargas de contenido visual (cards, listas)
- ✅ Usa **spinner** para acciones rápidas y operaciones breves
- ✅ El estado vacío debe tener un **llamado a la acción** útil
- ✅ Los mensajes de error deben decir **qué pasó** y **qué hacer**
- ✅ Usa `role="alert"` para errores urgentes, `role="status"` para info
- ✅ **Toasts** para feedback no bloqueante; **modal** para errores que requieren decisión
- ❌ No muestres solo "Error" o "No hay datos" sin contexto

🎉